

DDA3020 Homework 2

Due date: April 16, 2026

Instructions

- The **deadline** is **23:59, April 16, 2026**, please submit in time.
- The total score for this assignment is **100 points**.
- **Electronic submission:** Submit your solutions via Blackboard with the following files:

- One PDF file containing all written answers and programming results:

`DDA3020HW2_studentID_name.pdf`

- Two Python scripts for the programming question:

`HW2_name_Q2_1.py`, `HW2_name_Q2_2.py`

Replace `studentID` with your student ID and `name` with your name.

- For the programming question, your PDF must include: (i) the key code blocks you filled in, (ii) the numerical results, and (iii) a short analysis. The outputs shown in the PDF must match the submitted source code.
- The complete and executable code must be submitted. If you only report results in the PDF but do not submit the source code (`.py` or `.ipynb` file), you will receive **0 points** for the programming question.
- Answer the questions in English. Otherwise, you will lose half of the points.
- Collaboration policy: You need to solve all questions independently and collaboration between students is **NOT** allowed.

1 Written Problems (50 points)

1.1. (Decision Tree with Entropy, 25 points) Consider the following training set for predicting whether a student is likely to be overweight.

Table 1: Training set for overweight prediction

Stu_ID	Gender	Hyperlipidemia	Unhealthy Diet	Exercises	Overweight
1	boy	yes	yes	yes	yes
2	boy	yes	yes	no	yes
3	girl	no	yes	no	yes
4	boy	no	no	yes	no
5	girl	yes	yes	yes	yes
6	boy	no	yes	yes	no

Use **entropy** as the splitting criterion to construct a classification decision tree for predicting whether a student is overweight. Show your main computation steps clearly. When comparing candidate features at one node, choose the feature with the **smallest conditional entropy** (equivalently, the largest information gain).

- (4 points)** Compute the entropy of the root node.
- (8 points)** Compute the conditional entropy after splitting on each feature:

Gender, Hyperlipidemia, Unhealthy Diet, Exercises.

Then determine which feature should be selected as the **root node**.

- (6 points)** Continue splitting the non-pure child node(s) using the same criterion until all leaf nodes are pure, or no further split is possible. Clearly show the entropy computation for the selected split at each step.
- (4 points)** Draw the final decision tree.
- (3 points)** Use your final tree to predict the label of the following new student, and explain the decision path:

(Gender = girl, Hyperlipidemia = yes, Unhealthy Diet = no, Exercises = yes).

Solution:

- Root entropy.**

At the root, among the 6 samples, there are 4 positive samples (yes) and 2 negative samples (no). Hence, $H(D) = -\frac{4}{6} \log_2 \frac{4}{6} - \frac{2}{6} \log_2 \frac{2}{6} = -\frac{2}{3} \log_2 \frac{2}{3} - \frac{1}{3} \log_2 \frac{1}{3} \approx 0.9183$. (4 points)

(b) Conditional entropy for each feature.

1) Split on Gender

For Gender = boy, we have samples {1,2,4,6}, containing 2 yes and 2 no:

$$H(D_{\text{boy}}) = -\frac{2}{4} \log_2 \frac{2}{4} - \frac{2}{4} \log_2 \frac{2}{4} = 1.$$

For Gender = girl, we have samples {3,5}, containing 2 yes and 0 no:

$$H(D_{\text{girl}}) = 0.$$

Therefore, $H(D \mid \text{Gender}) = \frac{4}{6} \cdot 1 + \frac{2}{6} \cdot 0 = 0.6667$. (1.5 points)

2) Split on Hyperlipidemia

For Hyperlipidemia = yes, we have samples {1,2,5}, containing 3 yes and 0 no:

$$H(D_{\text{yes}}) = 0.$$

For Hyperlipidemia = no, we have samples {3,4,6}, containing 1 yes and 2 no:

$$H(D_{\text{no}}) = -\frac{1}{3} \log_2 \frac{1}{3} - \frac{2}{3} \log_2 \frac{2}{3} \approx 0.9183.$$

Therefore, $H(D \mid \text{Hyperlipidemia}) = \frac{3}{6} \cdot 0 + \frac{3}{6} \cdot 0.9183 \approx 0.4591$. (1.5 points)

3) Split on Unhealthy Diet

For Unhealthy Diet = yes, we have samples {1,2,3,5,6}, containing 4 yes and 1 no:

$$H(D_{\text{yes}}) = -\frac{4}{5} \log_2 \frac{4}{5} - \frac{1}{5} \log_2 \frac{1}{5} \approx 0.7219.$$

For Unhealthy Diet = no, we have sample {4}, containing 0 yes and 1 no:

$$H(D_{\text{no}}) = 0.$$

Therefore, $H(D \mid \text{Unhealthy Diet}) = \frac{5}{6} \cdot 0.7219 + \frac{1}{6} \cdot 0 \approx 0.6016$. (1.5 points)

4) Split on Exercises

For Exercises = yes, we have samples {1,4,5,6}, containing 2 yes and 2 no:

$$H(D_{\text{yes}}) = 1.$$

For Exercises = no, we have samples {2,3}, containing 2 yes and 0 no:

$$H(D_{\text{no}}) = 0.$$

Therefore, $H(D \mid \text{Exercises}) = \frac{4}{6} \cdot 1 + \frac{2}{6} \cdot 0 = 0.6667$. (1.5 points)

Thus,

$$H(D \mid \text{Hyperlipidemia}) \approx 0.4591$$

is still the smallest. Therefore, the root node should still be Hyperlipidemia. (2 points)

(c) Continue splitting.

For the branch $\text{Hyperlipidemia} = \text{yes}$, the subset is $\{1,2,5\}$. All labels are positive, so this branch is already pure and becomes a leaf: $\text{Hyperlipidemia} = \text{yes} \rightarrow \text{Overweight} = \text{yes}$.

(2 points)

For the branch $\text{Hyperlipidemia} = \text{no}$, the subset is $\{3,4,6\}$. Its labels are $\{\text{yes}, \text{no}, \text{no}\}$, so $H = -\frac{1}{3} \log_2 \frac{1}{3} - \frac{2}{3} \log_2 \frac{2}{3} \approx 0.9183$. (1 points)

Now evaluate the remaining features on this subset.

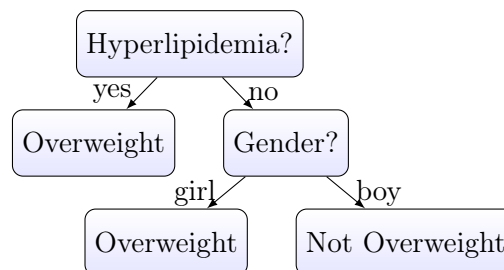
Split this subset on Gender

Within the subset $\{3,4,6\}$:

- $\text{Gender} = \text{girl}$: only sample 3, label yes $\Rightarrow H = 0$; (1 points)
- $\text{Gender} = \text{boy}$: samples 4 and 6, both labeled no $\Rightarrow H = 0$. (1 points)

Thus, $H(D_{\text{Hyperlipidemia=no}} \mid \text{Gender}) = \frac{1}{3} \cdot 0 + \frac{2}{3} \cdot 0 = 0$. So Gender perfectly splits this subset and should be selected. (1 points)

(d) Final decision tree. (4 points)



(e) Prediction for the new student.

For the new student, $(\text{Gender} = \text{girl}, \text{Hyperlipidemia} = \text{yes}, \text{Unhealthy Diet} = \text{no}, \text{Exercises} = \text{yes})$, we start from the root node. Since $\text{Hyperlipidemia} = \text{yes}$, we directly reach the leaf node

$\text{Overweight} = \text{yes}$. (2 points)

Therefore, the predicted label is yes. (1 points)

1.2. (Multiple Choice Questions for Neural Network, 9 points)

1. Suppose you design a deep multi-layer feedforward neural network, but you only use *linear* activation functions for every hidden layer. What is the expressive power of this network? (3 points)
 - A) It becomes a universal function approximator.
 - B) It is not more expressive than a standard linear regression model.
 - C) It can capture non-linear dependencies but cannot solve the XOR problem.
 - D) It performs better than a single-layer network with a non-linear activation.
2. What is the primary motivation for using Long Short-Term Memory (LSTM) networks over basic Recurrent Neural Networks (RNNs)? (3 points)
 - A) LSTMs have fewer parameters than basic RNNs, making them faster to train.
 - B) LSTMs process sequential data in parallel, solving the limited parallelization issue of RNNs.
 - C) LSTMs provide a solution to the vanishing gradients problem, making it easier to learn long-distance dependencies.
 - D) LSTMs replace the hidden state with a multi-head attention mechanism.
3. According to the lectures, which of the following is a major advantage of the Transformer architecture compared to Recurrent Neural Networks (RNNs)? (3 points)
 - A) Transformers enforce strict sequential processing of data.
 - B) Transformers allow for easier parallelization during training.
 - C) Transformers use sparse connections to reduce the number of parameters.
 - D) Transformers rely exclusively on basic fully-connected layers without positional encodings.

Solution:

1. B (3 points)

2. C (3 points)

3. B (3 points)

1.3. (CNN, 16 points) Consider the convolutional network defined by the layers below. The input shape is $60 \times 60 \times 3$ (regard it as an RGB image in a traffic sign recognition scenario) and the output is 8 neurons (classifying into 8 different traffic sign categories). A CNN-based classifier can be defined as:

$$\text{Conv5}(12) + \text{Maxpool}_2 + \text{Conv3}(16) + \text{FC8}$$

where

- Conv5(12): 12 filters with each size $5 \times 5 \times D$, where D is the depth of the activation volume at the previous layer, stride = 1, padding = 0;
- Maxpool₂: 2×2 filter, stride = 2, padding = 0;
- Conv3(16): 16 filters with each size $3 \times 3 \times D$, where D is the depth of the activation volume at the previous layer, stride = 1, padding = 1;
- FC8: A fully-connected layer with 8 output neurons.

Task: Compute the shape of activation map and the total number of parameters of each layer. Fill the results in the table.

NO.	Layer	Activation Shape	# Parameters	Mark
1	Input Layer	(60, 60, 3)	0	-
2	Conv5(12)			[2+2] pts
3	Maxpool2			[2+2] pts
4	Conv3(16)			[2+2] pts
5	FC8			[2+2] pts

Solution:

NO.	Layer	Activation Shape	# Parameters	Mark
1	Input Layer	(60, 60, 3)	0	-
2	Conv5(12)	(56, 56, 12)	912	[2+2] pts
3	Maxpool2	(28, 28, 12)	0	[2+2] pts
4	Conv3(16)	(28, 28, 16)	1744	[2+2] pts
5	FC8	(8) or (8,1) or (1,8)	100360	[2+2] pts

Calculation Breakdown (for grading reference only):

- **Layer 2 (Conv5):**

- Shape: $W = (60 - 5 + 2 \times 0)/1 + 1 = 56 \Rightarrow (56, 56, 12)$

- Params: $(5 \times 5 \times 3 + 1) \times 12 = 76 \times 12 = 912$

- **Layer 3 (Maxpool2):**

- Shape: $W = 56/2 = 28 \Rightarrow (28, 28, 12)$

- Params: 0 (pooling layers have no learnable parameters)

- **Layer 4 (Conv3):**

- Shape: $W = (28 - 3 + 2 \times 1)/1 + 1 = 28 \Rightarrow (28, 28, 16)$

- Params: $(3 \times 3 \times 12 + 1) \times 16 = 109 \times 16 = 1744$

- **Layer 5 (FC8):**

- Shape: (8)

- Params: Flattened input is $28 \times 28 \times 16 = 12544$. Total params = $(12544 + 1) \times 8 = 100360$

2 Programming (50 points)

You should write the key code you completed, the experimental results you obtained, and a brief analysis in the PDF file. Note that the results generated by your source code must exactly match those reported in the PDF.

2.1. (Decision Tree vs. Random Forest, 25 points) In this problem, you will compare a single `DecisionTreeClassifier` and a `RandomForestClassifier` from `sklearn` on the **Breast Cancer Wisconsin dataset**, which is a **binary classification** problem with 30 real-valued input features. A partially completed code template will be provided, and you need to fill in the missing blocks marked by `[TASK]`.

The goal of this exercise is two-fold:

- to help you understand how to train and evaluate tree-based models using `sklearn`, and
- to compare the performance of a single decision tree with that of an ensemble model (random forest).

Fixed Experimental Settings

For fairness and reproducibility, you must use the following settings exactly:

- Dataset: `load_breast_cancer()` from `sklearn.datasets`
- `random_state = 42`
- `test_size = 0.2`
- Use `stratify = y` when splitting the dataset
- For the Decision Tree:
 - `criterion = 'entropy'`
 - `max_depth = 3`
- For the Random Forest:
 - `criterion = 'entropy'`
 - `max_depth = 3`
 - `max_features = 'sqrt'`
 - test `n_estimators` in $\{10, 50, 100\}$

Your Tasks

You need to complete the Python code for the following steps:

(a) **(3 points) Data Loading and Splitting**

Complete the code to load the Breast Cancer dataset and split it into training and test sets according to the fixed settings above.

(b) **(6 points) Decision Tree Training and Evaluation**

Complete the code to build, train, and evaluate one `DecisionTreeClassifier`. Report both the training accuracy and the test accuracy.

(c) **(6 points) Random Forest Training and Evaluation**

Complete the code to build, train, and evaluate three `RandomForestClassifier` models with different numbers of trees:

$$n_estimators \in \{10, 50, 100\}.$$

Report the training accuracy and the test accuracy for each setting.

(d) **(3 points) Best Model Selection**

Compare all models and identify which one achieves the best **test accuracy**.

(e) **(7 points) Short Analysis**

Briefly discuss about your results and answer the following questions in your PDF report:

- Does Random Forest always outperform a single Decision Tree on the training set?
- Why can Random Forest often generalize better than a single tree?

Report Requirement

In your PDF, you must include:

- the key code blocks that you filled in,
- the printed training/test results for all required models, and
- a short comparison and discussion of the experimental results.

Solution:

With the required settings in the question, a correct implementation should produce the following results:

Dataset split (3 points)

Training size = 455, Test size = 114

Decision Tree (6 points)

Train Accuracy = 0.9670, Test Accuracy = 0.9474

Random Forest (6 points)

`n_estimators = 10` : Train Accuracy = 0.9692, Test Accuracy = 0.9561

`n_estimators = 50` : Train Accuracy = 0.9824, Test Accuracy = 0.9474

`n_estimators = 100` : Train Accuracy = 0.9846, Test Accuracy = 0.9474

Therefore, the model with the best test accuracy is

Random Forest with `n_estimators = 10`.

(3 points)

A concise analysis is:

- In this experiment, the Random Forest with 10 trees achieves the highest test accuracy, while larger values of `n_estimators` do not further improve the test performance.
- Random Forest does **not necessarily** outperform a single Decision Tree by a large margin in every experiment. Here, the best Random Forest only slightly outperforms the Decision Tree on the test set.

- Compared with a single Decision Tree, Random Forest often generalizes better because it combines multiple trees trained with randomness in sampling and feature selection, which usually reduces variance and improves robustness.
- The fact that the training accuracy increases from 10 to 50 to 100 trees, while the test accuracy does not improve accordingly, also reminds us that higher training accuracy does not always imply better generalization.

(7 points)

Note for grading. Since the experiment settings are fixed in the question, students' reported numerical results should match the required setup. If different random seeds, split strategies, or hyperparameter settings are used, the numerical results should not be treated as fully correct.

2.2. (Neural Network, 25 points) In Lecture 09, we discussed a core concept of deep learning: neural networks can be viewed as a way of learning features. The hidden layers transform the original input space into a new representation space where the data points, which might originally be non-linearly separable, become **linearly separable**.

In this section, you will empirically validate this theory. You will build a **Multi-Layer Perceptron (MLP)** from scratch using **PyTorch** to classify the classic **Breast Cancer Dataset** (a binary classification task with 30 real-world medical features).

To visualize the "Feature Learning" process, you will intentionally design the network with a **2-dimensional bottleneck hidden layer**. By extracting the outputs of this 2D layer and plotting them, you will witness how the neural network naturally separates the malignant and benign samples into two distinct clusters on a 2D plane.

Note: You are strictly forbidden from using high-level wrappers like `sklearn.neural_network.MLPClassifier`. You must build the `nn.Module` and the training loop manually using `PyTorch`.

Part I: Data Preparation (Provided)

The dataset contains 569 samples, each with 30 features. Before feeding data into a neural network, it is crucial to standardize the features. Please use the following code to load and preprocess your data:

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 1. Load the Breast Cancer dataset (30 features, binary target label)
data = load_breast_cancer()
X, y = data.data, data.target

# 2. Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Data Standardization
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 4. Convert NumPy arrays to PyTorch Tensors
X_train = torch.FloatTensor(X_train)
y_train = torch.FloatTensor(y_train).view(-1, 1) # Reshape to (N, 1)
X_test = torch.FloatTensor(X_test)
y_test = torch.FloatTensor(y_test).view(-1, 1)
```

Listing 1: Data Loading and Preprocessing

Part II: Model Construction & Training (Your Task)

You need to write the Python code for the following three steps:

Step A: Define the MLP Architecture

Create a Python class named `CustomMLP` that inherits from `nn.Module`. Your network must strictly follow this architecture:

- **Input Layer:** 30 features.
- **Hidden Layer 1:** 16 neurons, followed by a ReLU activation function.
- **Hidden Layer 2 (Bottleneck):** 2 neurons, **NO** activation function (keep it linear so we can plot unrestricted 2D coordinates).
- **Output Layer:** 1 neuron, followed by a Sigmoid activation function (to output a probability between 0 and 1).

Important requirement: Your `forward(self, x)` function must return **two** variables: the final prediction probability (`out`) AND the output of Hidden Layer 2 (`hidden_2d`).

Step B: Write the Training Loop

1. Instantiate your model.
2. Define the Loss function: Use Binary Cross Entropy Loss (`nn.BCELoss()`).

3. Define the Optimizer: Use Adam (`optim.Adam`) with a learning rate of `0.01`.
4. Train the model for **200 epochs**.
5. Inside the loop, perform the standard PyTorch training steps: Forward pass $\rightarrow$ Compute Loss $\rightarrow$ Zero Gradients $\rightarrow$ Backward pass $\rightarrow$ Optimizer Step.
6. Save the loss value of each epoch into a list and plot the Training Loss curve using `matplotlib`.

Step C: Evaluate the Model

After training, switch your model to evaluation mode (`model.eval()`). Disable gradient tracking using `torch.no_grad()`: and pass the `X_test` data through your model.

- Extract the final predictions. If the output probability is ≥ 0.5 , classify it as 1 (Benign); otherwise 0 (Malignant).
- Calculate and print the **Test Accuracy**. (*A properly trained model should easily achieve $> 95\%$ accuracy*).
- Ensure you save the 2D bottleneck output for `X_test` into a variable named `test_hidden_2d`.

Part III: Feature Space Visualization (Provided)

If you have successfully completed Part II, you now have a variable named `test_hidden_2d` containing the 2-dimensional coordinates mapped by your neural network for the test set.

Run the following provided code to visualize these learned features:

```
# Assuming 'test_hidden_2d' and 'y_test' are available from your evaluation step
hidden_features = test_hidden_2d.numpy()
labels = y_test.numpy().flatten()

plt.figure(figsize=(8, 6))

# Plot Malignant tumors (Label 0) in red
plt.scatter(hidden_features[labels==0, 0], hidden_features[labels==0, 1],
            color='red', label='Malignant (Class 0)', alpha=0.7, edgecolors='k')

# Plot Benign tumors (Label 1) in blue
plt.scatter(hidden_features[labels==1, 0], hidden_features[labels==1, 1],
            color='blue', label='Benign (Class 1)', alpha=0.7, edgecolors='k')

plt.title("2D Hidden Space Learned by MLP")
plt.xlabel("Hidden Neuron 1 Output")
plt.ylabel("Hidden Neuron 2 Output")
plt.legend()
plt.grid(True)
plt.show()
```

Listing 2: Feature Visualization

Grading Criteria

- **[10 points] Model Implementation:** Correctly build the MLP architecture (including the 2D bottleneck layer) and implement the training loop manually using PyTorch as specified in Part II.
- **[10 points] Code Execution:** The submitted code successfully runs without errors through the entire training and evaluation process, correctly computing the loss and test accuracy. And the test accuracy achieves at least 90%.
- **[5 points] Visualization & Observation:** Display the **Training Loss Curve** and the final **2D Hidden Space Scatter Plot** in your PDF report. In addition, briefly write down your findings or what you have learned from visualizing this 2D feature space (e.g., Have the two classes of data points become nearly "linearly separable"? How does this demonstrate the "feature learning" capability of neural networks?).

Solution:

The graph results should be similar to:

